

In the following sections, I'll call the scene the user is seeing the *current* scene. The scene that the current scene is sending information to or requesting information from is the *target* scene. The methods used to transfer information are all implemented in the `Scene` base class. Therefore, all scenes (which must inherit from the `Scene` base class) have access to these methods using `self.<method>()`.

Requesting Information from a Target Scene

To request information from any other scene, a scene makes a call to the inherited `request()` method, like this:

```
self.request(targetSceneKey, requestID)
```

This call allows the current scene to ask for information from a target scene, identified by its scene key (`targetSceneKey`). The `requestID` uniquely identifies the information you are asking for. The value used for a `requestID` would normally be a constant defined in a file like *Constants.py*. The call returns the requested information. A typical call would look like this:

```
someData = self.request(SOME_SCENE_KEY, SOME_INFO_CONSTANT)
```

This effectively says, “Issue a request to the `SOME_SCENE_KEY` scene asking for information identified by `SOME_INFO_CONSTANT`.” The data is returned and assigned to the `someData` variable.

The scene manager acts as an intermediary: it receives the call to `request()` and turns it into a call to `respond()` in the target scene. To make a target scene able to give information, you must implement a `respond()` method in that scene's class. The method should start like this:

```
def respond(self, requestID):
```

The typical code of a `respond()` method checks the value of the `requestID` parameter and returns the appropriate data. The data returned can be formatted in any way that the current scene and the target scene agree on.